

CMW RAG

Архитектура развёртывания

Оглавление

1. Хост	4
1.1 Запущенные сервисы	4
1.2 Незапущенные сервисы	4
2. Топология репозитория	5
2.1 Граф зависимостей	5
2.2 Схема соединений	6
2.3 Краткая таблица управления сервисами	6
3. CMW-Mosec (порт 7998)	8
3.1 Маршруты	8
3.2 Переменные среды активных моделей	8
3.3 Запуск	8
3.4 Альтернативные модели	9
4. ChromaDB (порт 8000)	10
4.1 Пуск / Стоп / Статус	10
4.2 Файл сервиса	10
4.3 Активная коллекция	10
4.4 Версионированные коллекции	10
5. RAG Gradio UI (порт 7860)	11
5.1 Два агента	11
5.2 Пуск / Стоп / Статус	11
5.3 Файл сервиса	11
5.4 Схема связей	11
5.5 Переменные среды	12
6. Интеграция с CMW Platform	15
6.1 Два экземпляра	15
6.2 Основной: конвейер обработки заявки	15
6.3 Второй / Лукойл: резюме документов	17

6.4 Pydantic-схемы	17
6.5 Category Enum	18
7. Синхронизация корпуса (MkDocs)	19
7.1 Systemd-таймер	19
7.2 Установка таймера	19
7.3 Внешний репозиторий	19
7.4 Версии корпуса	19
7.5 Ручной запуск	20
8. Инструкция по восстановлению	21
8.1 Требования к GPU	21
9. Соседние репозитории	23
9.1 cmw-mosec	23
9.2 cmw-vllm	23
10. Заметки	24

1. Хост

1.1 Запущенные сервисы

Сервис	Порт	Процесс	Статус
RAG Gradio UI	7860	<code>cmw-rag-app.service</code> (systemd)	Работает
CMW-Mosec	7998	<code>cmw-rag-mosec.service</code> (systemd)	Работает
ChromaDB	8000	<code>cmw-rag-chroma.service</code> (systemd)	Работает

1.2 Незапущенные сервисы

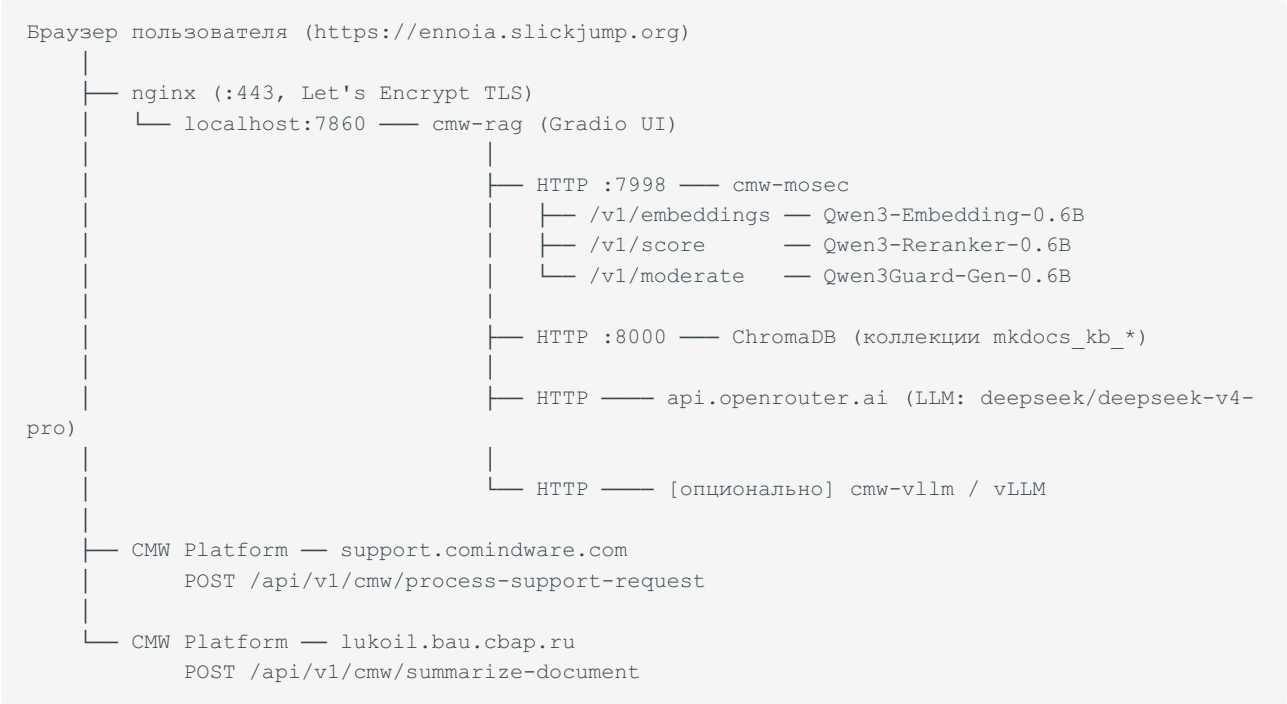
Отсутствуют — все основные сервисы активны.

2. Топология репозиториев

Развёртывание CMW RAG образуют три смежных репозитория. cmw-rag — центральный оркестратор; cmw-mosec обеспечивает локальный инференс; cmw-vllm — опциональная локальная замена OpenRouter.

Репозиторий	Роль	Точка входа	Порт	systemd-юнит	Статус
cmw-rag	RAG: Gradio UI + ChromaDB + интеграция с платформой	rag_engine/ api/app.py	: 7860	cmw-rag- app.service	☒ Работает
	Векторное хранилище ChromaDB	chroma run	: 8000	cmw-rag- chroma.service	☒ Работает
cmw-mosec	Инференс эмбеддингов, ранжировщика, модератора	cmw-mosec serve -- foreground	: 7998	cmw-rag- mosec.service	☒ Работает
cmw-vllm	Опционально: локальная LLM (замена OpenRouter)	cmw-vllm start	8001	cmw-rag- vllm.service	☐ Не развёрнут

2.1 Граф зависимостей



2.2 Схема соединений

Откуда	Куда	Протокол	Переменные среды в cmw-rag
cmw-rag (эмбединги)	cmw-mosec /v1/ embeddings	HTTP POST	MOSEC_EMBEDDING_ENDPOINT, EMBEDDING_MODEL
cmw-rag (ранжировщик)	cmw-mosec /v1/ score	HTTP POST	MOSEC_RERANKER_ENDPOINT, RERANKER_MODEL
cmw-rag (модератор)	cmw-mosec /v1/ moderate	HTTP POST	GUARD_MOSEC_ENDPOINT
cmw-rag (LLM)	OpenRouter	HTTP POST	OPENROUTER_BASE_URL, OPENROUTER_API_KEY
cmw-rag (LLM)	cmw-vllm (опционально)	HTTP POST	VLLM_BASE_URL, VLLM_API_KEY
cmw-rag (векторный поиск)	ChromaDB	HTTP	CHROMADB_HOST, CHROMADB_PORT

Все systemd-юниты находятся в `cmw-rag/systemd/` и устанавливаются симлинком в `~/config/systemd/user/`.

2.3 Краткая таблица управления сервисами

Сервис	systemctl --user ...	Логи
Mosec (:7998)	start/stop/restart/status cmw-rag-mosec	journalctl --user -u cmw-rag-mosec -f
ChromaDB (:8000)	start/stop/restart/status cmw-rag-chroma	journalctl --user -u cmw-rag-chroma -f
RAG UI (:7860)	start/stop/restart/status cmw-rag-app	journalctl --user -u cmw-rag-app -f
Корпус (таймер)	enable/disable/start/stop/status cmw-rag-corpus-sync.timer	journalctl --user -u cmw-rag-corpus-sync -f
Корпус (oneshot)	start cmw-rag-corpus-sync.service	systemctl --user list-timers --user
vLLM (:8001, опционально)	start/stop/restart/status cmw-rag-vllm	journalctl --user -u cmw-rag-vllm -f

vLLM опционален — нужен только если `DEFAULT_LLM_PROVIDER=vllm` в `cmw-rag/.env`. Порт 8000 занят ChromaDB на этом хосте; установите `VLLM_PORT=8001` в `cmw-vllm/.env`.

Первоначальная установка каждого сервиса:

```
ln -sf /путь/к/cmwr-ag/systemd/<файл-сервиса> ~/.config/systemd/user/  
systemctl --user daemon-reload  
systemctl --user enable --now <имя-сервиса>
```

Предварительное условие (первый раз на хосте):

```
loginctl enable-linger $USER
```

Быстрый статус всех сервисов:

```
systemctl --user status cmwr-ag-mosec cmwr-ag-chroma cmwr-ag-app cmwr-ag-vllm
```

3. CMW-Mosec (порт 7998)

Один процесс Mosec обслуживает несколько моделей. Маршруты регистрируются в зависимости от переменных среды.

Репозиторий: `github.com/cmw-team/cmw-mosec`.

3.1 Маршруты

Маршрут	Worker	Модель	Проверка
<code>POST /v1/embeddings</code>	EmbeddingWorkerV2	<code>Qwen/Qwen3-Embedding-0.6B</code>	☒ Возвращает эмбединги
<code>POST /v1/score</code>	ScoreWorkerV2	<code>Qwen/Qwen3-Reranker-0.6B</code>	☒ Возвращает оценки
<code>POST /v1/rerank</code>	RerankWorkerV2	<code>Qwen/Qwen3-Reranker-0.6B</code>	☒ Ошибка выполнения
<code>POST /v1/moderate</code>	GuardWorkerV2	<code>Qwen/Qwen3Guard-Gen-0.6B</code>	☒ Возвращает модерацию

3.2 Переменные среды активных моделей

```
ACTIVE_EMBEDDING_MODEL=Qwen/Qwen3-Embedding-0.6B
ACTIVE_RERANKER_MODEL=Qwen/Qwen3-Reranker-0.6B
ACTIVE_GUARD_MODEL=Qwen/Qwen3Guard-Gen-0.6B
SERVER_PORT=7998
DEVICE=auto
DTYPE=float32
BATCH_SIZE=32
HF_TOKEN=<токен-huggingface>
```

3.3 Запуск

Production (systemd user service):

```
systemctl --user start/stop/restart/status cmw-rag-mosec
journalctl --user -u cmw-rag-mosec -f
```

Разработка (вручную):

```
cd cmw-mosec
source .venv/bin/activate
cmw-mosec serve --foreground
```

3.4 Альтернативные модели

Тип	Модели
Эмбединги	<code>ai-forever/FRIDA</code> (1536, ~4 ГБ), <code>Qwen/Qwen3-Embedding-0.6B</code> (1024, ~2 ГБ), <code>Qwen/Qwen3-Embedding-4B</code> (2560, ~12 ГБ), <code>Qwen/Qwen3-Embedding-8B</code> (4096, ~22 ГБ)
Ранжировщики	<code>DiTy/cross-encoder-russian-msmarco</code> (~2 ГБ), <code>BAAI/bge-reranker-v2-m3</code> (~2 ГБ), <code>Qwen/Qwen3-Reranker-0.6B</code> (~2 ГБ), <code>Qwen/Qwen3-Reranker-4B</code> (~12 ГБ), <code>Qwen/Qwen3-Reranker-8B</code> (~22 ГБ)
Модераторы	<code>Qwen/Qwen3Guard-Gen-0.6B</code> (~4 ГБ), <code>Qwen/Qwen3Guard-Gen-4B</code> (~10 ГБ), <code>Qwen/Qwen3Guard-Gen-8B</code> (~20 ГБ)

.....

4. ChromaDB (порт 8000)

Векторное хранилище для поиска в RAG.

Управляется как **systemd user service** (`cmw-rag-chroma.service`). Автоматически перезапускается при сбоях и после перезагрузки.

4.1 Пуск / Стоп / Статус

```
systemctl --user start/stop/restart/status cmw-rag-chroma
journalctl --user -u cmw-rag-chroma -f
```

4.2 Файл сервиса

`systemd/cmw-rag-chroma.service` в репозитории `cmw-rag`. Устанавливается через СИМЛИНК В `~/.config/systemd/user/`.

```
ExecStart=%h/cmw-rag/.venv/bin/chroma run --host 0.0.0.0 --port 8000 --path %h/cmw-rag/data/chromadb_data
```

4.3 Активная коллекция

`mkdocs_kb_qwen06b_linux_qwen_default_instruct_chunk_768` (1024 измерения, соответствует Qwen3-Embedding-0.6B).

4.4 Версионированные коллекции

Коллекция	Содержимое
<code>..._v5</code>	Документация платформы v5
<code>..._v6</code>	Документация платформы v6

При смене модели эмбеддингов (иной размерности) требуется создать новую коллекцию и переиндексировать данные.

5. RAG Gradio UI (порт 7860)

Точка входа: `rag_engine/api/app.py`.

Репозиторий: `github.com/cmw-team/cmw-rag` (pushurl: `arterm-sedov`).

5.1 Два агента

Агент	Путь	Особенности
Support Assistant	<code>/</code>	Полный интерфейс: выбор версии, панели метаданных, SRP (план решения), экспорт чата, MCP-инструменты (<code>ask_comindware</code> , <code>get_knowledge_base_articles</code>)
KB Assist	<code>/kb_assist</code>	Тот же агент, минимальный интерфейс (только чат и строка ввода, без панелей и выбора версии, <code>skip_srp=True</code>)

Оба агента используют один и тот же обработчик LangChain (`chat_with_metadata`). KB Assist лишь скрывает панели метаданных и отключает SRP.

Управляется как **systemd user service** (`cmw-rag-app.service`). Зависит от ChromaDB и Mosec (`After=cmw-rag-chroma.service cmw-rag-mosec.service`).

5.2 Пуск / Стоп / Статус

```
systemctl --user start/stop/restart/status cmw-rag-app
journalctl --user -u cmw-rag-app -f
```

5.3 Файл сервиса

`systemd/cmw-rag-app.service` в репозитории `cmw-rag`.

```
Environment=PYTHONPATH=%h/cmw-rag
ExecStart=%h/cmw-rag/.venv/bin/python rag_engine/api/app.py
```

5.4 Схема связей



5.5 Переменные среды

```

# LLM
OPENROUTER_API_KEY=<ключ>
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
DEFAULT_LLM_PROVIDER=openrouter
DEFAULT_MODEL=deepseek/deepseek-v4-pro

# Эмбединги
EMBEDDING_PROVIDER_TYPE=mosec
EMBEDDING_MODEL=Qwen/Qwen3-Embedding-0.6B
MOSEC_EMBEDDING_ENDPOINT=http://localhost:7998/v1/embeddings

# Ранжирование
RERANK_ENABLED=true
RERANKER_PROVIDER_TYPE=mosec
RERANKER_MODEL=Qwen/Qwen3-Reranker-0.6B
RERANKER_TIMEOUT=60.0
RERANKER_MAX_RETRIES=3
MOSEC_RERANKER_ENDPOINT=http://localhost:7998/v1/score

# Модерация
GUARD_ENABLED=true
GUARD_BLOCK_THRESHOLD=unsafe
GUARD_PROVIDER_TYPE=mosec
GUARD_MOSEC_ENDPOINT=http://localhost:7998/v1/moderate

# ChromaDB
CHROMADB_PORT=8000
CHROMA_CLIENT_HOST=localhost
CHROMA_SERVER_BIND=0.0.0.0
CHROMADB_COLLECTION=mkdocs_kb_qwen06b_linux_qwen_default_instruct_chunk_768
CHROMADB_PERSIST_DIR=~/.cmw-rag/data/chromadb_data

# Параметры поиска
TOP_K_RETRIEVE=20
TOP_K_RERANK=10
CHUNK_SIZE=768
CHUNK_OVERLAP=75
RETRIEVAL_MULTIQUERY_ENABLED=true
RETRIEVAL_MULTIQUERY_MAX_SEGMENTS=4
RETRIEVAL_MULTIQUERY_SEGMENT_TOKENS=448
RETRIEVAL_QUERY_DECOMP_ENABLED=true
RETRIEVAL_QUERY_DECOMP_MAX_SUBQUERIES=4

# Gradio
GRADIO_SERVER_NAME=0.0.0.0
GRADIO_SERVER_PORT=7860
GRADIO_LOCALE=ru

# Опционально: ключ Google API (запасной провайдер Gemini)
# GOOGLE_API_KEY=<ключ>
# Опционально: vLLM (замена OpenRouter)
# VLLM_BASE_URL=http://localhost:8000/v1
# VLLM_API_KEY=EMPTY

# Интеграция с CMW Platform — основной (агент поддержки)
CMW_BASE_URL=https://support.comindware.com
CMW_LOGIN=<логин>
CMW_PASSWORD=<пароль>
CMW_TIMEOUT=30
CMW_API_KEY=<ключ-апи>

# Интеграция с CMW Platform — второй / Лукойл (резюме)
CMW2_BASE_URL=<url-второй-платформы>

```

```
CMW2_LOGIN=<логин>
CMW2_PASSWORD=<пароль>
CMW2_TIMEOUT=30
CMW2_API_KEY=<ключ-api>

# Рассуждения LLM
LLM_REASONING_ENABLED=true
LLM_REASONING_MAX_TOKENS=1200
LLM_REASONING_EXCLUDE_FROM_RESPONSE=true
```

6. Интеграция с CMW Platform

RAG взаимодействует с двумя экземплярами Comindware Platform через FastAPI и YAML-конвейеры. Оба работают по принципу fire-and-forget: прочитать запись — запустить агента в фоне — записать результат.

6.1 Два экземпляра

Экземпляр	Назначение	Файл конфигурации	Префикс env
Основной	Поддержка: заявка → ответ RAG → запись ответа	cmw_platform.yaml	CMW_
Второй (Лукойл)	Резюме документов: скачать → LLM → записать	cmw_platform_secondary.yaml	CMW2_

6.2 Основной: конвейер обработки заявки

Endpoint: POST /api/v1/cmw/process-support-request

Схема: {"request_id": "string"}

Процесс:

CMW Platform (support.comindware.com)

POST /api/v1/cmw/process-support-request {request_id}

▼

RAG FastAPI

1. Аутентификация по X-API-Key (если задан CMW_API_KEY)

2. Чтение записи из systemSolution.Requests

Поля: name, Description, currentBuild, browserDetails

3. Сборка markdown-запроса по шаблону

4. Ответ 200: {success: true, message: "Request fetched, agent started at ..."}

5. Фоновый поток → агент LangChain (ask_comindware_structured)

6. Преобразование ответа агента в выходные поля

7. Запись ответа в systemSolution.agent_responses

(связь с исходной заявкой через support_request)

▼

Ответ записан в CMW Platform

YAML-конфигурация конвейера (cmw_platform.yaml):

```

pipeline:
  input:
    application: systemSolution
    template: Requests
    attributes:
      support_case_title: name
      support_case_question: Description
      product_version: currentBuild
      user_browser: browserDetails

  request_template: |
    ---
    - product version: {product_version}
    - user browser: {user_browser}
    ---
    # {support_case_title}
    {support_case_question}

  output:
    application: systemSolution
    template: agent_responses
    record_attribute: support_request
    linked_template: Requests

```

Сопоставление ответа агента с атрибутами (`mapping.py`):

- `answer` — окончательный ответ
- `question_for_agent` — исходный запрос с YAML-шапкой
- `agent_thinking` — результат исследования SGR
- `issue_area` — категория из `category_enum`
- `kb_articles` — процитированные статьи

6.3 Второй / Лукойл: резюме документов

Endpoint: `POST /api/v1/cmw/summarize-document`

Схема: `{"request_id": "string"}`

Процесс:

```
CMW Platform (lukoil.bau.cbap.ru, приложение: ArchitectureManagement)
| POST /api/v1/cmw/summarize-document {record_id}
▼
RAG FastAPI
| 1. Аутентификация по X-API-Key (если задан CMW2_API_KEY)
| 2. Чтение записи из ArchitectureManagement.Zaprosinarazrabotky
|   Поля: Commerpredloshenie (документ), prompt (запрос пользователя)
| 3. Загрузка документа → извлечение текста
| 4. Ответ 200: {success: true, message: "Начата обработка данных"}
| 5. Фоновый поток → create_summary_agent (LangChain)
|   Агент с веб-поиском, системный промпт из YAML
| 6. Преобразование резюме в HTML
| 7. Запись резюме в атрибут summary записи
▼
Саммари записано в CMW Platform
```

YAML-конфигурация (`cmw_platform_secondary.yaml`):

```
pipeline:
  input:
    application: ArchitectureManagement
    template: Zaprosinarazrabotky
    attributes:
      document_file: Commerpredloshenie
      user_prompt: prompt

  output:
    application: ArchitectureManagement
    template: Zaprosinarazrabotky
    summary_attribute: summary
    summary_as_html: true

  system_prompt: |
    Ты — профессиональный бизнес-ассистент. Твоя задача — составлять
    краткие и информативные резюме деловых документов.
    ...
```

6.4 Pydantic-схемы

```

class ProcessSupportRequest(BaseModel):
    request_id: str

class SummarizeDocumentRequest(BaseModel):
    request_id: str

class ProcessResult:
    success: bool
    message: str | None = None
    error: str | None = None

class HTTPResponse(BaseModel):
    success: bool
    status_code: int
    raw_response: dict | str | None = None
    error: str | None = None
    base_url: str

class APIResponse(BaseModel):
    response: Any | None = None
    success: bool | None = None
    error: str | None = None

class RequestConfig(BaseModel):
    base_url: str
    login: str
    password: str
    timeout: int = 30

```

6.5 Category Enum

Категории обращений загружаются из `cmw_platform.yaml` → `category_enum`.
 Список получается из CMW Platform скриптом `scripts/fetch_issue_areas.py`.
 Примеры: `account`, `api`, `deployment`, `email`, `performance`, `integrations` и др.

7. Синхронизация корпуса (MkDocs)

Синхронизирует документацию из внешнего MkDocs-репозитория в ChromaDB для индексации RAG.

7.1 Systemd-таймер

Компонент	Значение
Таймер	<code>cmw-rag-corpus-sync.timer</code> — включён и активен
Расписание	Каждые 6 часов в 00:00, 06:00, 12:00, 18:00 UTC (+10 мин случайной задержки)
Сервис	<code>cmw-rag-corpus-sync.service</code> — oneshot, запускается по таймеру
Команда	<code>sync_mkdocs_corpus.py --index --corpus all</code>

7.2 Установка таймера

```
cp systemd/cmw-rag-corpus-sync.{service,timer} ~/.config/systemd/user/  
systemctl --user enable --now cmw-rag-corpus-sync.timer
```

7.3 Внешний репозиторий

Параметр	Значение
Remote	<code>github.com/arterm-sedov/cbap-mkdocs-ru</code>
Ветка	<code>platform_v6</code>
Sparse path	<code>phpkb_content_rag</code>
Локальная копия	<code>.reference-repos/cbap-mkdocs-ru</code>

7.4 Версии корпуса

Версия	Путь в репозитории	Суффикс коллекции ChromaDB
v5	<code>phpkb_content_rag/798-platform_v5</code>	<code>_v5</code>
v6	<code>phpkb_content_rag/896-platform_v6</code>	<code>_v6</code>

Скрипт использует `git sparse-checkout` для загрузки только каталога `phpkb_content_rag`, затем индексирует новые и изменённые файлы. Если изменений нет — индексация пропускается.

7.5 Ручной запуск

```
python rag_engine/scripts/sync_mkdcs_corpus.py --index --corpus all  
python rag_engine/scripts/sync_mkdcs_corpus.py --index --corpus v6
```

8. Инструкция по восстановлению

1. **Установить Python 3.11+** и создать виртуальное окружение для каждого проекта.
2. **Установить зависимости:** `pip install -e .` в cmw-mosec, `pip install -r rag_engine/requirements.txt` в cmw-rag.
3. **Установить ChromaDB:** `pip install chromadb`.
4. **Настроить .env :** в cmw-mosec (модели, порт, токен HuggingFace) и в cmw-rag (ключи LLM, учётные данные платформы).
5. **Установить systemd-сервисы** — см. `systemd/` в cmw-rag — симлинк в `~/.config/systemd/user/` и `systemctl --user enable --now` для каждого
6. **Запустить Mosec:** `systemctl --user start cmw-rag-mosec` — при первом запуске скачает модели (~8 ГБ видеопамати GPU).
7. **Запустить ChromaDB:** `systemctl --user start cmw-rag-chroma`.
8. **Построить индекс:** запустить синхронизацию корпуса для наполнения коллекций.
9. **Запустить RAG UI:** `systemctl --user start cmw-rag-app`.
10. **Настроить webhook'и CMW Platform:** - `POST http://<host>:7860/api/v1/cmw/process-support-request` (support.comindware.com) - `POST http://<host>:7860/api/v1/cmw/summarize-document` (lucoil.bau.cbap.ru)
 - Оба принимают `{"request_id": "..."}` и опциональный заголовок `X-API-Key`.
11. **Проверить:**
 - `curl http://localhost:7998/v1/embeddings -X POST -d '{"input":"test","model":"Qwen/Qwen3-Embedding-0.6B"}'`
 - `curl http://localhost:7998/v1/moderate -X POST -d '{"input":"test"}'`
 - `curl http://localhost:8000/api/v1/heartbeat`
 - `curl http://localhost:7860/api/v1/cmw/process-support-request -X POST -d '{"request_id":"test"}'`
 - Открыть `http://localhost:7860` и `http://localhost:7860/kb_assist` в браузере.

8.1 Требования к GPU

Mosec с тремя активными моделями требует около 8 ГБ видеопамати GPU (эмбеддинг — 2 ГБ, ранжировщик — 2 ГБ, модератор — 4 ГБ). Все обработчики работают на одном устройстве через пакетную обработку Mosec.

Несоответствие версий драйвера: на хосте установлены два пакета `nvidia-driver` (590 — модуль ядра, 595 — пользовательские библиотеки). PyTorch выводит

"Can't initialize NVML", но это **некритично** — CUDA-вычисления и инференс работают корректно.

vLLM (если развёрнут) требует отдельного GPU — модель `openai/gpt-oss-20b` нуждается в ~24 ГБ видеопамяти. Может работать на том же GPU, что и Mosec, при наличии свободной памяти, или на отдельном хосте.

9. Соседние репозитории

9.1 cmw-mosec

Параметр	Значение
Репозиторий	<code>github.com/cmw-team/cmw-mosec</code> (pushurl: <code>arterm-sedov</code>)
Точка входа	<code>cmw_mosec.cli:cli</code> (Click CLI)
Модуль сервера	<code>cmw_mosec.v2.dynamic_server</code>
Конфигурация моделей	<code>config/models.yaml</code>
Использование	Активно: эмбединги, оценки, модерация на этом хосте

Запуск: `systemctl --user start cmw-rag-mosec` (обёртка вокруг `cmw-mosec serve --foreground`). Читает `ACTIVE_*_MODEL`, запускается на `SERVER_PORT`.

9.2 cmw-vllm

CLI-утилита для управления inference-серверами vLLM (совместимы с API OpenAI). Может использоваться как локальная замена OpenRouter.

Параметр	Значение
Репозиторий	<code>github.com/cmw-team/cmw-vllm</code> (pushurl: <code>arterm-sedov</code>)
Точка входа	<code>cmw_vllm.cli:cli</code> (Click CLI)
Реестр моделей	<code>cmw_vllm/model_registry.py</code>
Активная модель (если запущен)	<code>openai/gpt-oss-20b</code>
Порт по умолчанию	8000 (используйте 8001 — конфликт с ChromaDB)
Статус	Не развёрнут. Требуется отдельный GPU.
systemd-юнит	<code>cmw-rag-vllm.service</code> (в <code>systemd/</code> <code>cmw-rag</code>)

```
# .env: VLLM_MODEL=openai/gpt-oss-20b VLLM_PORT=8001 VLLM_HOST=0.0.0.0
systemctl --user start cmw-rag-vllm
```

RAG подключается через `VLLM_BASE_URL=http://<host>:8001/v1` и `DEFAULT_LLM_PROVIDER=vllm`.

10. Заметки

- Маршрут `/v1/rerank` в Moses зарегистрирован, но возвращает ошибку выполнения. RAG использует `/v1/score`, который работает корректно.
- vLLM может служить локальным LLM-бэкендом вместо OpenRouter при развёртывании на отдельном хосте со свободным GPU и портом.
- При смене модели эмбеддингов (иной размерности) следует создать новую коллекцию ChromaDB и переиндексировать данные.